开源项目架构附录

陈星强 · github.com/chenxingqiang · 每页包含项目介绍与模块级系统架构



阅读约定

自上而下阅读。虚线框是架构分区，内层实线框是子系统，小盒子是具体模块。

蓝色实线是主数据流，紫色虚线是回退或反馈，不把源码目录本身当成架构层。

TraceSmith — 系统架构 · GPU Trace 端到端数据流

Application → _tracesmith.so → Capture / State / Format → Replay → GPU backends → trace outputs

主数据流　虚线框 = 架构分区
回退 / 反馈　盒内 = 模块 + 职责

项目介绍

TraceSmith 是面向 AI 编译器、深度学习框架与 GPU 驱动工程师的跨平台性能分析和确定性回放系统。

C++17 核心以低开销采集执行事件，写入紧凑 SBT 轨迹，并导出时间线用于分析、调试与问题复现。

APPLICATION · 用户入口

- **C++ Application** — 嵌入业务进程
- **Python Application** — 嵌入业务进程
- **C++ CLI** — tracesmith; profile · record · view
- **Python CLI** — tracesmith-cli; export · analyze · replay
- **External UI** — Perfetto UI; Chrome tracing

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — python/src/bindings.cpp; pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — 跨设备采集
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — 集合通信
- **ClockCorrelator** — 时钟对齐
- **CommAnalysis** — 通信矩阵

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU 内存
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer; xctrace · msprof

State Module

- **GPUStateMachine** — 状态分析
- **InstructionStream** — 依赖图
- **TimelineBuilder** — 事件排序
- **TimelineViewer** — ASCII 渲染
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

Replay Module

- **ReplayEngine** — 回放编排
- **StreamScheduler** — 操作顺序
- **OperationExecutor** — 执行回放
- **DeterminismChecker** — 结果比对
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType; KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — 二进制写入
- **SBTReader** — 二进制读取
- **.sbt** — header + events; counters

Configuration

- **ProfilerConfig** — 缓冲与溢出
- **ReplayConfig** — 模式与速度
- **StackCaptureConfig** — 调用栈策略
- **NCCLTrackerConfig** — 集合通信策略

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — 断点与回放
- **tracesmith-gdbserver** — v0.10.0

回放

GPU BACKENDS · 系统接口

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — 分布式
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — 主机调用栈
- **Perfetto SDK** — 轨迹导出

OUTPUTS · Trace 产物

- **trace.sbt** — 二进制格式
- **trace.json** — Perfetto JSON
- **trace.perfetto** — Protobuf
- **graph.dot** — Graphviz
- **ASCII Timeline** — 终端
- **Comm Matrix** — 热力图 JSON

主链：GPU event → RingBuffer → SBT/state → replay 或 Perfetto。Cluster、配置与 GDB 是并列能力。

YiRage (YPK) — 系统架构 · RuntimeFusion 数据流

Engine owns schedule + KV; RF owns fusion identity; BackendRegistry searches and executes the same backend

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

YiRage 是面向大模型推理的多后端算子编译与运行时系统，核心能力是计算图超优化和 RuntimeFusion。

系统离线搜索并验证后端特定融合计划，再将 FusionCapsule 注入 vLLM 或 SGLang 的逐步解码流程。

INFERENCE ENGINE · 调度与 KV 所有者

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up → down
- **Native Fallback** — skip → engine native ops

SEARCH & PLAN · 离线搜索与预热

- **superoptimize** — KN / TB graph search
- **FusionPlan** — μGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — 可选分布式搜索
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API 与 BACKEND MANAGER

- **KNGraph / TBGraph** — transpiler → kernels
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — 可选

BACKEND IMPLEMENTATIONS · 后端实现

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS · 硬件目标

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

回退

主链：decode metadata → RF.step → FusionCapsule → BackendRegistry → 同一后端与硬件。

JAX-sklearn (xlearn) — 五层架构 · 透明加速数据流

xlearn/__init__.py injects proxies; xlearn/_jax dispatches to JAX or the original sklearn estimator

主数据流　　虚线框 = 架构分区
回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

JAX-sklearn 在保持 scikit-learn 调用接口不变的前提下，透明地以 JAX 加速受支持的估计器。
运行时代理依据算法、数据规模与硬件平台选择 JAX 或原生 sklearn，并保持输入输出兼容。

USER CODE · sklearn 兼容接口

- **import xlearn as sklearn** — 100% call surface
- **Estimators** — linear_model · cluster · decomposition · preprocessing
- **Calls** — fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__** — 惰性导入子模块
- **_auto_jax_accelerate_module** — 替换估计器类
- **AcceleratorRegistry** — _accelerator.py
- **create_intelligent_proxy** — _proxy.py
- **monkey_patch_estimator** — 安装代理类

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy** — fit-time dispatch
- **create_accelerated_estimator** — _accelerator.py
- **_universal_jax.py** — JIT · vmap
- **original_class** — sklearn 回退
- **_gp.py** — Gaussian process
- **_mlp.py** — neural net
- **_svm.py** — SVM kernels
- **_tree.py** — tree models

回退

DATA MANAGEMENT · 数组与策略

- **_data_conversion.py** — NumPy ↔ JAX · dtype · shape
- **_config.py** — enable_jax · jax_platform
- **jax_auto_threshold** — complexity >= 1e8
- **detect_hardware** — install command · platform probe

HARDWARE · JAX 后端

- **CPU** — jax-cpu · jaxlib
- **CUDA GPU** — jax-gpu · jax-cuda11
- **TPU** — jax-tpu
- **Apple Metal** — jax-metal · float32 · experimental

主链：sklearn 调用 → 注入代理 → JAX 估计器。小数据、缺算子或执行失败时回到 original_class。

Secret-Learn — 六层架构 · 隐私计算数据流

secretlearn API → FL / SS / SL placement → JAX acceleration → SecretFlow devices

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

Secret-Learn 以 sklearn 兼容接口提供联邦学习、秘密分享和拆分学习三种隐私机器学习模式。

系统结合 JAX 加速与 SecretFlow 安全设备，使工作流能够明确控制明文、秘密份额和中间激活的驻留位置。

APPLICATION · 多方使用场景

- **Healthcare** — 跨院协作 · 患者隐私
- **Finance** — 跨行分析 · 欺诈检测
- **IoT / Edge** — 边缘智能 · 分布式学习
- **Research** — 基因组 · 多机构
- **Government** — 普查分析 · 跨部门数据

SKLEARN API · 191 个算法

- **Linear Models** — 39 — Ridge · Lasso · SGD
- **Ensemble** — 18 — RF · XGBoost
- **Clustering** — 14 — KMeans · DBSCAN
- **Decomposition** — 14 — PCA · NMF · ICA
- **SVM** — 7 — SVC · SVR
- **Neural Networks** — 4 — MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees · Naive Bayes

PRIVACY MODES · 执行位置

FEDERATED LEARNING · 联邦学习

- **Local PYU** — 数据留在本地
- **HEU Aggregate** — 安全聚合
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING · 秘密分享

- **SPU MPC** — 密文计算
- **ABY3 / CHEETAH** — MPC 协议
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING · 拆分学习

- **Model Split** — 跨参与方切分
- **HEU Activations** — 保护中间激活
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

JAX ACCELERATION · xlearn 桥接

- **JIT / XLA** — 跟踪执行
- **jax.vmap** — 批处理与自动微分
- **xlearn Backend** — JAX-sklearn 桥
- **Smart Fallback** — 回退原生 sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

SECURE DEVICES · SecretFlow 设备

SPU

- **MPC** — ABY3 · CHEETAH
- **Ciphertext** — 不暴露明文

HEU

- **Schemes** — Paillier · CKKS
- **Aggregation** — 梯度保护

TEE

- **Enclaves** — Intel SGX · AMD SEV
- **Occlum** — 证明

PYU

- **Party Local** — 数据归属
- **JAX Local** — 本地明文计算

INFRASTRUCTURE · 运行时基础设施

- **SecretFlow** — Ray distributed · gRPC · DP-SGD
- **JAX / XLA** — CUDA / cuDNN · TPU runtime
- **sklearn** — NumPy · SciPy

主链：同一次 sklearn 调用选择 FL、SS 或 SL，再落到 PYU、HEU、SPU 或 TEE。

YouTube-DNN-RecSys — 系统架构 · 召回数据流

Python / Spark samples → TFRecords → dnn.py user tower → Spark or Python prediction

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

YouTube-DNN-RecSys 使用 TensorFlow 与 Spark 实现 YouTube 推荐系统中的候选召回阶段。

系统将用户行为历史转换为 TFRecords，训练用户与物品向量，并为在线服务生成 Top-K 召回候选。

DATA PROCESSING · 数据处理层

Python 数据处理

- **data2tfrecords.py**
  生成训练数据
- **TFRecords**
  训练与评估集
- **read_tfrecords.py**
  数据集读取

Scala + Spark 数据处理

- **MakeDataOne.scala**
  分布式生成
- **MakeDataTwo.scala**
  TFRecords output
- **HDFS**
  数据集存储

STORAGE · 存储层

- **data/tfrecords/**
  train/train.tfrecords
  evaluation/evaluation.tfrecords
- **data/checkpoints/**
  ckpt/checkpoint
  ckpt/eval · modelpath/

MODEL TRAINING · dnn.py

- **Input**
  visit_items_index × 5
  continuous_features × 16
- **Item Embedding**
  500 items × 16
- **Average Pooling**
  历史向量
- **DNN Tower**
  Dense 64 + ReLU
  Dropout · Dense 32 + ReLU
  user vector 16
- **Softmax Output**
  500 classes
  cross entropy · Top-K

评估

PREDICTION · 预测服务层

- **PredictUserVector.scala**
  SavedModel
  embedding_average → user_vector
- **ItemEmbeddingPredictor.scala**
  item embedding table
  FeatureBuilder
- **load_dnn_model.py**
  SavedModel load
  在线与批量推理

UTILITIES · 工具与辅助层

- **tensor_board.py**
  TensorBoard
- **feature_column.py**
  特征列
- **self_defined_network_layer.py**
  自定义网络层
- **Spark Base**
  BaseSparkLocal.scala
  BaseSparkOnline.scala
- **Configuration**
  vars.prod.properties
  vars.sit.properties

主链：行为历史 → TFRecords → embedding average pool → user vector → item table → Top-K。

Research CLI — 系统架构 · Agent 请求数据流

packages/cli → packages/core routing → tools / MCP / research modules → model providers

主数据流　虚线框 = 架构分区
回退 / 反馈　盒内 = 模块 + 职责

项目介绍

Research CLI 是终端优先的 AI Agent 工作流，将交互会话连接到模型、工作区工具和 MCP 服务。

核心路由负责上下文与模型供应商选择，研究模块覆盖论文写作、文献管理、数据分析和投稿准备。

CLI FRONTEND · packages/cli

- **research.tsx** — 交互终端
- **commands/** — mcp.ts · skills.tsx · hooks.tsx
- **ui/commands/research** — arxivCommand · paper · investigate · submit
- **acpSession.ts** — ACP session · stdio transport

CORE ORCHESTRATION · packages/core

会话

- **researchChat.ts** — researchRequest.ts
- **research-config-manager.ts** — 学术配置
- **config / memory** — 上下文

路由

- **modelRouterService** — routing/modelRouterService.ts
- **defaultStrategy** — routingStrategy.ts
- **classifierStrategy** — gemma · numerical
- **compositeStrategy** — approval · fallback · override

TOOLS 与 RESEARCH MODULES

Workspace

- **edit.ts · grep.ts** — filesystem changes
- **shell / ls** — 本地执行

MCP

- **mcp-client.ts** — 服务会话
- **mcp-client-manager.ts** — 资源路由

analysis/

- **bibliography-manager** — citations
- **experiment-code-generator** — Python · R · Julia
- **research-data-analyzer** — 统计与机器学习

bibliography/

- **arxiv-mcp-client** — arXiv search
- **google-scholar-client** — scholar search
- **literature-investigator** — keyword-generator · reference-classifier

submission/

- **latex-manager** — project compile
- **journal-matcher** — 期刊匹配
- **arxiv-latex-extractor** — 模板提取

回路

MODEL PROVIDERS · 模型供应商

- **Gemini / Vertex**
- **OpenAI**
- **Anthropic**
- **Baidu ERNIE**
- **DeepSeek**
- **Qwen**
- **Groq / Mistral** — Cohere · Moonshot

EMBEDDED RUNTIMES · 嵌入运行时

- **packages/sdk** — 库嵌入
- **packages/a2a-server** — agent executor · HTTP task API
- **vscode-ide-companion** — IDE companion
- **packages/devtools** — 开发工具

主链：prompt → research.tsx → modelRouterService → provider；工具和研究模块返回观察结果。

Questions-Gen — 系统架构 · 三阶段训练数据流

Question data → Qwen3-14B SFT → GRPO reward → DeepSeek-R1 distillation → validation → publish

主数据流　　虚线框 = 架构分区
回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

Questions-Gen 是基于 Qwen3-14B 的数学竞赛题生成与评估系统，采用分阶段训练策略。

训练链路结合监督微调、GRPO 奖励优化、DeepSeek-R1 蒸馏、质量验证以及 Hugging Face 和 Ollama 部署。

TRAINING DATA · 训练数据混合

- **Historical** — 50% — QuestionDataPreparer.py
- **Conditional Variants** — 30% — data_preparer.py
- **Innovative Types** — 20% — real_math_data_processor.py

Stage 1 · SFT · 监督微调

- **Qwen3-14B** — 基座模型
- **QuestionsGenTrainer.py** — src/ + questions_gen/core/trainer.py
- **stage1-fp-merged** — Hugging Face

Stage 2 · GRPO · 奖励优化

RewardCalculator.py

- **Difficulty** — 0.4 difficulty
- **Novelty** — 0.3
- **Rigor** — 0.2
- **Diversity** — 0.1

优化

- **GRPO Groups** — 8 problems/group
- **NoveltyConstraint.py** — 重复约束
- **stage2-merged-16bit** — variation generation

质量

Stage 3 · Distill · 知识蒸馏

- **DeepSeekTeacher.py** — DeepSeek-R1
- **Teacher Signals** — 难度与严谨性 / 创新与教学价值
- **final-merged-16bit** — Hugging Face

VALIDATION · questions_gen/validation

- **model_validator.py** — 单模型
- **batch_validator.py** — algebra · calculus / geometry · number theory
- **quality_evaluator.py** — clarity · difficulty / originality

DEPLOYMENT · 发布与接口

- **main_cli.py** — validate · batch · quality
- **hf_utils.py** — three published stages
- **ollama_manager.py** — 本地推理
- **examples/** — quick_demo.py / demo_model_validation.py

主链：混合题库 → SFT → GRPO reward 与 novelty → DeepSeek-R1 蒸馏 → 验证 → Hugging Face / Ollama。